

Federated PHC Supply Chain Platform — Working Roadmap

Team: **A — Harshit** (Android), **B — Shivakumar** (Backend/data), **C — Sudhanva** (Forecasting), **D — Ananya** (Agents/orchestration + dashboard)

Availability: Shivakumar & Sudhanva building from **20th-25th**. Harshit & Ananya join from **26th onward**. Everyone is building their part from scratch with no prior expertise in it.

Assumption: this roadmap targets a ~10-day window ending around the 29th, based on the original brief. Adjust the back half if your actual deadline is different.

Honest flag up front: two people building the backend + forecasting foundation for 6 days, then all four building the Android app, 3 AI agents, and dashboard in ~4 days — with nobody having done Android or agentic AI before — is a tight runway. The two things that make it survivable: Harshit and Ananya doing a bit of self-study *before* the 26th (no build time needed, just prep), and keeping the Android app to one core screen instead of a polished multi-screen product. Both are baked into the plan below.

Stage 1 — 20th to 25th: Shivakumar & Sudhanva

They take the two tracks that don't depend on the Android UI or the agent layer existing yet — backend/data and forecasting — so nothing they build is blocked waiting on Harshit or Ananya.

20th (Day 1)

- Both: a short FastAPI + PostgreSQL crash course, get a "hello world" API running locally
- Together: draft the full data model — `phc(id, name, district, lat, lon)`, `drug(id, name)`, `stock(phc_id, drug_id, quantity, expiry_date, updated_at)`, `prescription(id, phc_id, drug_id, qty, doctor_id, patient_ref[optional], timestamp)`, `transfer_request(id, from_phc, to_phc, drug_id, qty, reason, status, approved_by)`. This becomes the contract Harshit and Ananya build against later, so it's worth getting mostly right even if rough

21st (Day 2)

- Shivakumar: CRUD endpoints for `phc`, `drug`, `stock`
- Sudhanva: quick primer on time-series basics (moving average → exponential smoothing) using `pandas`

22nd (Day 3)

- Shivakumar: `prescription` + `transfer_request` endpoints; start the synthetic multi-PHC dataset generator (8-12 PHCs, a few weeks of fake prescription history)
- Sudhanva: baseline forecasting model against early synthetic data as it becomes available

23rd (Day 4)

- Shivakumar: `get_nearby_phcs(phc_id, radius)` lookup (haversine distance); finish the dataset generator, planting a stock-out scenario, a near-expiry surplus, and a district-wide demand spike
- Sudhanva: add the nearby-PHC-weighted signal to forecasting; stock-out threshold logic

24th (Day 5)

- Shivakumar: expiry-flag logic on stock; deploy the backend somewhere reachable (a local network is fine for now)
- Sudhanva: early-warning spike detector; validate forecasts against the planted scenarios

25th (Day 6) — handoff day

- Both: write a short, clear handoff note — endpoint list with example requests/responses, how to run the forecasting model, where the synthetic dataset lives. This is what lets Harshit and Ananya start building on the 26th instead of spending their first day reverse-engineering what exists
- Buffer time to catch up on anything slipped

Checklist — Stage 1 (by 25th)

- ☐ Data model finalized and documented
 - ☐ Backend endpoints working: stock, prescriptions, transfer requests
 - ☐ Synthetic dataset built with stock-out, expiry, and spike scenarios planted
 - ☐ Nearby-PHC lookup function working
 - ☐ Forecasting model (own + nearby-weighted) working on synthetic data
 - ☐ Stock-out + expiry-flag logic working
 - ☐ Early-warning spike detector working
 - ☐ Handoff note written for Harshit & Ananya
-

Before the 26th — self-study (no build time needed)

- **Harshit:** a short Kotlin + Jetpack Compose "first app" tutorial, plus a quick intro to Room (offline local storage) — just enough to not start from zero
- **Ananya:** Google AI Studio's function-calling/tool-use quickstart, and a skim of what "agents" means here (a model calling backend endpoints as tools)

This costs no project time, just prep — worth doing given how tight the second half is.

Stage 2 — 26th onward: all four

26th (Day 7)

- Harshit: Android Studio project set up; build the prescription entry screen wired to Shivakumar's real API (skip offline sync for now, add later if time allows)
- Ananya: Google AI Studio project set up; get one Gemini function-calling agent working end-to-end against a single real backend endpoint, as a proof of concept before building all three
- Shivakumar & Sudhanva: unblock Harshit/Ananya as integration issues surface, keep improving the dataset and thresholds

27th (Day 8)

- Harshit: transfer-request status view; start the offline queue (Room) if on track
- Ananya: build out all three agents (forecasting, procurement, alert), each returning a short reasoning field; wire them to real endpoints
- Whole team: first full pipeline test — prescription → forecast → flag → transfer request

28th (Day 9)

- Harshit: one-tap approve/reject UI for incoming transfer requests; polish
- Ananya: dashboard — stock, bed/staff placeholders, alerts feed, impact counter
- Shivakumar & Sudhanva: bed/staff tables, expiry-triggered transfers wired alongside shortage-triggered ones, final threshold tuning
- Whole team: run the full demo scenario end-to-end, fix what breaks

29th (Day 10)

- Morning: final integration pass, freeze feature work
- Afternoon: rehearse the demo (1-2 live AI Studio calls, everything else cached/replayed), assign presenters, prepare a fallback recording in case of live API issues

Checklist — Stage 2 (by 29th)

- ☐ Android: prescription entry + status view working end-to-end against the real backend
 - ☐ All 3 agents working with reasoning output, wired to real endpoints
 - ☐ Full pipeline demo-able: prescription → forecast → flag → transfer request → dashboard
 - ☐ One-tap approval flow working
 - ☐ Dashboard with impact counter live
 - ☐ Demo rehearsed, presenter roles assigned, fallback ready
-

AI Studio credit / rate-limit plan (applies throughout Stage 2)

1. One key, one owner (Ananya), usage tracked daily
2. Mock first — real calls only once integration starts (27th onward)
3. Trigger-based calls only, never polling
4. Cache and replay real responses for repeated dev/testing and dry runs
5. Flash-tier model for agent orchestration; save heavier models only if a step truly needs it
6. Fix the exact live-call count for demo day in advance and rehearse that count